

01 – Zusammenfassung

- „automatisierte Abrechnungen“ wird erwähnt, aber keine Info zur rechtlichen/infrastrukturellen Absicherung (z. B. UID-Rechnungen, Reverse Charge, Mahnlogik) – viel zu pauschal.
 - „Fernwartung“ wird genannt, aber kein Hinweis auf OTA-Mechanismen, Sicherheitsprotokolle oder Zugriffskontrollen. Wie wird vermieden, dass Fernsteuerung kompromittiert wird?
-

02 – Ziele

- „DSGVO-konforme Kameras zur Verhinderung von Einbruch“ – Widerspruch: Einsatz zur Verhinderung ist präventiv, DSGVO erlaubt nur reaktive Verarbeitung. => „Ständige Überwachung“ verletzt aus meiner Sicht zu 99% der Fälle DSGVO (bin kein Jurist). Muss man auf jeden Fall im Antrag argumentieren.
 - „Barrierefreiheit“ ist als technisches Ziel aufgeführt, aber kein einziger Punkt beschreibt, wie das technisch umgesetzt werden soll (Screenreader? akustisches Feedback?).
-

03/04 – Warum bestehende Systeme nicht ausreichen

- „Integration professioneller Audiohardware“ – bleibt völlig unklar, welche Schnittstellen benötigt werden (USB, Dante, Thunderbolt?) und was „Integration“ bedeutet (nur Monitoring? Fernsteuerung? Firmware-Updates?).
 - „API-gesteuerte Fernwartung physischer Geräte“ – ist bei Mac Minis z. B. nicht out-of-the-box machbar, ohne zusätzliche Softwareagenten oder MDM-Systeme. Fehlt jeglicher Hinweis auf Implementierungsebene.
-

4.1 – Architekturübersicht

- MQTT + RabbitMQ parallel: kein technischer Grund genannt, warum zwei Message Broker nötig sind. Das ist architektonisch schwer wartbar – Split muss begründet werden.
- Elasticsearch + Prometheus + Grafana + Metabase: Monitoring und BI komplett vermischt. Elasticsearch ist eine Such- und Analyse-Engine und passt da nicht rein bzw. würde ich Monitoring und BI trennen.
- MongoDB + PostgreSQL + S3 → Overkill: 3 Speichertypen ohne Aufteilung nach Domain. Wozu S3, wenn keine Dateispeicherung beschrieben ist?

4.2 – Microservices

- „Insurance-Service“ → unklar, wie Integration mit Versicherern funktioniert. Gibt es APIs? Wird Schaden automatisch erkannt? Komplett abstrakt.
- „Monitoring-Service: Kameraüberwachung, Sensorstatus, Audiohardware“ → drei komplett unterschiedliche Datenquellen in einem Service – kein Microservice, sondern Monolith.
- „Payment-Service: Stripe/PayPal“ – kein Wort zu Währungsunterstützung, Rechnungslogik, Abbruchbehandlung, Webhook-Sicherheit.

4.3 – DevOps & CI/CD

- „Security-Scans“ wird genannt, aber keine Tools, keine Policies, kein Schwellenwert – reines Buzzword.
- Staging- und Production-Umgebungen auf AWS – keine Info, wie Rollouts passieren (Canary? Blue/Green?), keine Backups erwähnt.

05/06 – Warum keine fertige Lösung funktioniert

- „Zugangssteuerung in Echtzeit“ – kein Protokoll genannt, keine Aussage zur Latenz, Retry-Strategie oder Failover.
- „Versicherungskonforme Schadensmeldungen mit automatischer Belegerfassung“ – technisch unklar, ob OCR, manuelle Eingabe oder Fotoanhang. Gar keine Prozessbeschreibung.

07 – AI-Integration

- „Live-Sicherheitsanalysen aus Kamerastreams durch Objekterkennung“ – völlig unklar, wie DSGVO und Latenz unter einen Hut gebracht werden.
- „automatisierte Klassifikation von Schadensbildern“ – kein Dateninput, kein Modelltraining, keine Trainingsdatenherkunft erklärt. Nicht einmal High-Level plausibilisiert.
- Integration in bestehende Architektur nicht erklärt – nur Buzzwords („PyTorch als separater Microservice“), kein Schema, keine Ressourcenplanung.

08 – Fazit

- „Maximale Automatisierung“ – aber kein Wort zu Recovery-Strategien, Monitoring-Metriken oder Alert-Kaskaden.
 - Kein Hinweis auf Security by Design: Keine OWASP, keine Auth-Flows, keine Angriffsmodelle.
 - „SaaS global einsetzbar“, aber nichts zu Lokalisierung, Zeitzonen, Sprache, Datenschutzrechten außerhalb der EU.
-

09 – Umsetzungsplan MVP

- Zutrittsmodul: MQTT mit realer Türöffnung → Risiko extrem hoch, keine Redundanz oder Notfallmechanik beschrieben (z. B. Offline-Access-Codes, Zeitfenster-Token etc.).
- Monitoring: MongoDB für Time-Series ist suboptimal – kein Hinweis auf Performance bei hoher Datenrate.
- Kameras: „nur lokal gespeichert“ → wer betreibt die lokale Infrastruktur? Was passiert bei Diebstahl?
- Fernwartung: Keine Authentifizierung oder Zugriffskontrolle erklärt. Wer darf rebooten? Wie wird verhindert, dass ein Container während Nutzung neugestartet wird?

Allgemein

Sprachlich generell klingt der Antrag technisch, aber einige Formulierungen sind aus meiner Sicht unklar bis irreführend, wenn man sie genau analysiert.

z.B.: „Zugangssteuerung in Echtzeit erfordert direkte Gerät-zu-Cloud-Kommunikation“

Warum der Satz problematisch ist:

1. „Echtzeit“ ist technisch übertrieben oder missverständlich.
 - Echtzeit heißt in IT „100% garantierte Reaktion innerhalb definierter maximaler Zeit“ (z. B. 100 ms, bei einem Auto, wo Verzögerung = Tod wäre) – das gilt für Türsysteme meist nicht, sondern nur generelle „niedrige Latenz“.
 - Bei Netunterbrechung würde keine Echtzeit funktionieren, was gefährlich wäre (keine Türöffnung mehr möglich?).
2. „Direkte Gerät-zu-Cloud-Kommunikation“ ist oft gar nicht erwünscht.

- In der Praxis wird aus Sicherheits- und Stabilitätsgründen meist ein lokales Gateway oder Edge-Service eingesetzt.
- Direkte Kommunikation jedes Geräts mit der Cloud erfordert: Zertifikatsverwaltung, stabile Internetverbindung, OTA-Updatefähigkeit – sehr aufwändig.

3. Fehlt: Redundanz / Offline-Szenario.

- Ein Zugangssystem darf nicht „cloud-only“ funktionieren, sonst ist Zutritt bei Netzausfall nicht mehr möglich.
- Auch DSGVO-technisch ist es kritisch, wenn jedes Türereignis über öffentliche Netze geht.

Grafiken

Ist zwar wahrscheinlich Teil des Architekturplans, aber zumindest Sequenzdiagramme ohne die technischen Komponenten sind sicher Plus Punkte, um das System einfach verständlicher zu machen z.B.:

